

Final Report

Team 35

Abstract

As Large Language Models have become an integral part of our lives, aiding us in nearly every sphere of our everyday routines. From ChatGPT to LLaMa and Bard, numerous models have been established that are breaking limits in multiple fields. As the developments made using LLMs are breaking ground at light year speed, the improvements to LLMs are not very far behind. As a part of the High Prep Statement: AI Agent 007: Tooling up for Success organized by the Indian Institute of Technology, Madras, in collaboration with DevRev, we explore another improvement to Large Language Models, arming the models with the power to choose from a toolbox consisting of specific tools to help answer queries. It has been shown that many of the tool-augmented LLMs still mostly rely on closed LLM APIs. We attempt to build on open-source LLMs with minimal human intervention to achieve performance which is competitive with closed LLMs. Our code and data can be found here.¹

1. Introduction

As language models become more and more important in various spheres of our lives, pushing the boundaries of what these LLMs can achieve becomes crucial. Large Language Models have seamlessly integrated into our daily lives, transforming the way we interact with technology. Moreover, as LLMs continue to advance, we too, explore another such advancement for Large Language Models.

As a part of the High Prep Statement: AI Agent 007 - Tooling up for Success, organized by the Indian Institute of Technology, Madras, in collaboration with DevRev, we wish to further improve on the capabilities of open sourced LLMs. Armed with a set of specific tools, we wish to answer user-based queries in a step-wise manner using the tools provided, as well as, their arguments. Thus, our focus shifts to the realm of domain-specific question answering, where precision, context-awareness, and even the ordering within the answer becomes incredibly important.

¹[Link to code can be found here](#)

1.1. The Quest for Optimal User Interaction

As we come to terms with addressing user queries, the major objective is to compile a toolbox that the language model could use to facilitate a smooth conversation between the user and AI.

Our idea to solve the problem relies on how well we can decipher user queries, aptly mapping them to the appropriate tools, and defining the arguments necessary for their execution. This ensures that our responses are tailor-made for each particular query and the set of tools provided.

1.2. Domain-Specific Question Answering

In domain-specific question answering, precise tool selection emerges as a critical challenge, which forces us to have an acute understanding of the intricacies inherent to the domain in question.

Further, the dynamic toolbox, which can have a tool added, modified, or removed adds a layer of complexity. Thus, we require a fine balance between the selection of the tools, and deciphering the user queries to provide near-perfect answers.

1.3. Navigating the Landscape of LLMs

As and when we wish to increase the exposure of our closed LLMs, entrusting it with newer capabilities, security and robustness risks become apparent.

The journey through closed LLM APIs, while offering a streamlined pathway to enhanced capabilities, simultaneously raises concerns regarding data privacy, system integrity, and the overall resilience. There is a level of opacity that threaten the very foundations of security.

In recognition of these inherent risks associated with closed LLM APIs, mitigating these risks becomes imperative. While we cannot get rid of these risks completely, we wish to find a way around it. Thus, we wish to veer away from closed closed LLMs, and try figuring out an open-source answer to this.

2. Problem Statement

A Language Model L has a set of tools T and is given a user query Q . The model is supposed to answer Q using the existing tools, the relevant arguments for the tools chosen,

and the composition of the tools. Moreover, the set of the tools T is dynamic, and new tools could be added, or existing tools could be modified or deleted. The model should be able to handle all such cases gracefully.

3. Literature Review

The domain of domain-specific question answering and the orchestration of tools to address user queries represent a challenging area that has garnered considerable attention. Several research efforts and practical implementations have explored similar facets, providing insights into effective strategies and methodologies.

3.1. Background

Large Language Models (LLMs), such as GPT-3.5 and GPT-4 [31], are advanced AI models designed for text understanding and generation. GPT-4 excels in language tasks and image processing, achieving human-level performance on exams like the Uniform Bar Exam. Despite improvements, it has limitations, including no real-time learning and occasional reasoning errors.

LLaMa [43] challenges the idea that larger models necessarily lead to better performance, with models ranging from 7B to 65B parameters. LLaMa outperforms GPT-3 in various benchmarks, emphasizing common sense reasoning, question answering, and more. It addresses issues like toxicity and bias, evaluates carbon footprint, and highlights the importance of open access.

Augmented Language Models (ALMs) [29] address LLM limitations by combining reasoning and external tools. ALMs use diverse reasoning methods and tools integration, focusing on iterative LM calling and information retrieval strategies. Efforts to teach ALMs include few-shot prompting, fine-tuning, and reinforcement learning, with benefits in truthfulness, uncertainty reduction, interpretability, and enhanced capabilities. Ethical concerns revolve around the trustworthiness of ALM predictions using external tools. The approach involves leveraging large language models to create a logic layer or program atop retrieved APIs for query handling. This facilitates API sequence generation and logic incorporation, offering potential solutions for complex tasks. The modular strategy breaks down tasks, allowing recursion and proposing programs on generated API call sequences, including error handling for exceptions [43].

3.2. Data Augmentation using LLMs

To enhance the capabilities of language models (LMs) for query answering, data augmentation plays a crucial role, encompassing tool and question-answer (Q/A) generation. Large Language Models (LLMs) contribute significantly to both domains, expanding and diversifying datasets.

3.2.1. Tool Generation

LLMs proficiently generate synthetic tools, expanding the model's toolset. This involves simulating diverse tool functionalities, creating new tools with varying parameters and use cases. Strategies and considerations are detailed in papers such as [4] and [21], showcasing LLMs' effectiveness in tool generation.

3.2.2. Q/A Generation

Data augmentation extends to diverse Q/A pairs for comprehensive LM training. Leveraging LLMs, adept at paraphrasing and simulating linguistic styles, facilitates nuanced query and answer creation. Recent research optimizes LLMs for competitive tool manipulation, achieving up to a 90% performance boost in leading models [46]. In low-resource settings, fine-tuning smaller models with labeled examples from teacher LLMs enhances downstream performance significantly [17]. Pretrained LLMs contribute to online query policy learning, automating query formulation and enriching information from diverse sources [6].

3.2.3. Advanced Techniques

Several advanced techniques optimize LLMs for diverse applications. Self-Instruct enhances instruction-following, achieving a 33% improvement on Super-Natural Instructions [44]. Multi-agent automates training data generation for tool-augmented models, saving 98% compared to manual annotation [22]. The Prompting Framework (PF) efficiently manages LLM interactions, addressing limitations and showcasing applications in Integrated Application Platforms and LLM-based Agents. Tools like Prompt and Optimizer for Prompts, including OpenPrompt and Human-Prompt, enhance LLM flexibility and standardize human-friendly design [25].

In comparisons, LLM-SH surpasses LLM-RSTR and LLM-LNG in language compatibility, with strong support for OpenAI APIs. Regarding capabilities and features, LLM-SH exhibits strong compatibility, while LLM-LNG demonstrates excellent compatibility, particularly with interactive LLMs. These advancements collectively contribute to the evolving landscape of LM capabilities for query answering and beyond. To address the challenge of enhancing the capabilities of a language model (LM) for query answering using existing tools, data augmentation plays a pivotal role. This augmentation involves two key aspects: tool generation and question-answer (Q/A) generation. Large Language Models (LLMs) prove to be instrumental in both these domains, contributing to the expansion and diversification of the dataset.

3.3. Retrievers

The "Retriever" component in OpenQA systems [50] includes three main approaches: Sparse Retriever (TF-

IDF, BM25), Dense Retriever (Representation-based, Interaction-based, or a combination), and Iterative Retriever (Multi-step reasoning). Document post-processing is crucial, with methods such as neural Passage Rankers (e.g., R3), Paragraph Selectors (e.g., DS-QA), and time-aware re-ranking modules, although recent OpenQA systems tend to integrate trainable retrievers, potentially eliminating the need for a dedicated post-processing module.

ColBERT [18], or "Contextualized Late Interaction over BERT," is a neural network model designed for information retrieval tasks. Leveraging BERT's contextualized embeddings, ColBERT introduces a late interaction mechanism for dynamic and context-aware document ranking. Its architecture, based on BERT, excels in capturing nuanced language dependencies, making it effective in large-scale retrieval scenarios. ColBERT's versatility allows it to be combined with traditional techniques like TF-IDF for reranking, enhancing precision by assessing term importance based on both frequency in a document and rarity across a corpus. TF-IDF, a common reranking technique, refines initial document rankings by prioritizing terms' importance and distinctiveness in the context of a broader document collection.

Listwise rerankers based on large language models (LLM) are the zero-shot state-of-the-art. Various techniques are explored, including leveraging LLMs for generating synthetic datasets (InPars [4], Promptagator [11]), using LLMs directly as retrievers or rerankers (SGPT [30], UPR [38], PRP [37]), and treating LLMs as black boxes for ranking (RankGPT [41], LRL [28], RankVicuna [34]). This work [48] addresses limitations in listwise rerankers based on Large Language Models (LLMs), proposing GPT-free listwise rerankers that outperform GPT-based models in passage retrieval. The authors emphasize the inadequacy of existing pointwise training datasets for listwise rerankers, advocating for high-quality listwise training data. The method employs a sliding window strategy and highlights the significance of fine-tuning listwise-ranking LLMs. Results show superior effectiveness, reaching 97% with 5k high-quality training datapoints. Model size positively correlates with effectiveness, and even the smallest listwise reranker surpasses GPT-3 and GPT-3.5-based models. Ablation studies confirm the effectiveness of GPT-free LLMs and showcase the comparable performance of fine-tuning with a query likelihood-based approach. The study provides valuable insights into the benefits of listwise rerankers, offering a reproducible alternative to GPT-dependent models in information retrieval.

3.4. LLMs for answering

3.4.1. LLMs for API Calls

The approach utilizes various large language models (LLMs) to coordinate sub-tasks through APIs, efficiently distributing tasks among diverse LLMs. Inspired by [40],

one strategy employs an LLM as a planner, while another leverages latent language representations for skill induction and planning, focusing on API selection within the context [39]. The study by [15] explores breaking down high-level tasks into actionable steps without training, assessing execution and correctness through text completion and autoregressive trajectory correction. Gorilla [33] enhances LLMs by enabling interaction with external APIs, excelling in API functionality accuracy and hallucination reduction compared to GPT-4, highlighting adaptability to API documentation changes. ToolLLM [36] employs a three-step process, utilizing ChatGPT for instruction generation, a depth-first search-based decision tree for solution paths, and a neural retriever for automatic API selection, introducing metrics like pass rate and win rate for evaluation. This comprehensive approach showcases the potential of LLMs in orchestrating complex tasks through API interactions, with each referenced work contributing unique insights and methodologies.

3.4.2. Frameworks

ToolAlpaca [42] introduces a framework equipping compact language models with generalized tool-use abilities by constructing a diverse toolset using large language models (LLMs). The resulting corpus, containing instances from 426 tools across 50 categories, demonstrates high proficiency and empowers compact language models for generalized tool-use through fine-tuning. LangChain [1] is a sophisticated framework designed for context-sensitive applications utilizing language models, enabling the development of reasoning-enabled and context-aware applications. TaskMatrix.AI [23] envisions an AI ecosystem connecting foundation models, multimodal conversational foundation models, and diverse off-the-shelf models and systems via APIs for versatile task completion. The architecture includes an API Platform, API Selector, and Action Executor, employing Reinforcement Learning with Human Feedback (RLHF) to optimize models and foster iterative improvements based on user feedback. Together, these frameworks showcase the potential of language models in diverse applications, ranging from generalized tool use to context-sensitive and multimodal task completion.

3.4.3. Q/A Approaches

The proposed approach for addressing complex questions involves treating the problem as a Question-Answering (QA) task, where the language model formulates precise questions directed at an external retriever or semantic model, ensuring flexibility and scalability in handling modifications to the toolset. This strategy is discussed in [45]. Another approach, presented in [49], breaks down complex questions into sub-questions sequentially, generating follow-up questions to external systems or APIs based

on the obtained answers, refining the model’s understanding iteratively. Text Modular Networks (TMNs) [19] introduces a framework for complex question-answering by decomposing questions into sub-questions answered by existing QA models, showcasing versatility, robustness, and interpretable reasoning. Successive Prompting for Decomposing Complex Questions [13] proposes a modular approach involving question decomposition and answering stages, achieving a 5.1% F1 improvement over the state-of-the-art model on the DROP dataset. These approaches highlight the effectiveness of modular systems in handling complex reasoning tasks and question-answering.

3.4.4. External Tools as Problem Solvers

The papers [27], [32], and [9] adopt a coding paradigm for problem-solving, utilizing external tools (APIs) akin to coding functions. They emphasize adaptability, sequential tool-use for automated reasoning, and the ability to incorporate new tools or modifications. Chameleon [27] employs an LLM-based planner to generate plans for sequentially executing tools to accomplish complex reasoning tasks. The model assembles a sequence of tools based on input queries, planning task instructions, module descriptions, demonstration examples, and constraints. The generated plan guides the sequential execution of modules, updating cached information at each time step.

3.4.5. Fine-tuning and Performance Improvement

Recent developments in large language models (LLMs) have prompted researchers to explore fine-tuning approaches that specialize models with generalization capabilities in a parameter-efficient manner. In-context learning (ICL) [5] conditions the model on natural language instructions and demonstrations, allowing it to predict subsequent instances of a task without extensive fine-tuning. Parameter-efficient fine-tuning (PEFT) [24] minimally updates a few parameters, demonstrating computational efficiency compared to ICL. Another facet involves enhancing the performance of generated API call sequences by implementing a majority voting mechanism inspired by [16], which aggregates outputs from diverse methods for more robust outcomes. Additionally, a verifier, inspired by [10], evaluates the effectiveness of the generated sub-task sequences in achieving the desired task, contributing to confidence and reliability.

3.4.6. Bonus Task Approaches

The [7] approach involves leveraging large language models to create a logic layer or program atop retrieved APIs for query handling. This not only facilitates the generation of API sequences but also allows the incorporation of logic, offering potential solutions for the bonus section. [20] adopts a modular strategy to solve complex tasks by break-

ing them down, with the capability to employ recursion, which aligns with one of the proposed approaches for the bonus section. The idea is to propose a program on the generated API call sequence, introducing logic such as error handling (e.g., try-except in Python) to address exceptions that may arise from API calls, a relevant consideration for the bonus section.

3.5. Retrieval-Augmented Generation (RAG)

[3] introduces Retrieval Augmented Generation (RAG), an AI framework enhancing Large Language Model (LLM)-generated responses by grounding the model on external knowledge sources. RAG ensures access to up-to-date, reliable facts, reducing hallucinations and fostering trust in the model’s claims. In [8], the impact of RAG on LLMs is investigated through the Retrieval-Augmented Generation Benchmark (RGB). The evaluation highlights challenges, including noise robustness, negative rejection, information integration, and counterfactual robustness. Current LLMs face difficulties in handling noise, rejecting irrelevant information, integrating across documents, and identifying errors, emphasizing the need for cautious and well-designed RAG applications to ensure accurate and reliable responses..

4. Experimental Setup

4.1. Tool Generation

Aside from the 9 tools that were given to us in the original Problem Statement, we generated another 83 tools. 8 of the tools were generated using GPT-4 by prompting it to generate tools that may cover the various stages of the software development life cycle. These were followed by 71 tools, which were parsed from the APIs in the Github Repository of the paper [22]. Finally, we generated 4 additional tools to tackle the Bonus Problems, more details of which are given in 6.

4.2. Dataset

We utilize GPT-3.5 and GPT-4 for data augmentation along with manual human intervention to generate tools for smoother functionality. We further created additional data points with different levels of utilization in terms of the tools and arguments that will be used to resolve each query. Figure 1 describes the number of examples available utilizing each tool and Table 1 contains the dataset statistics for tools and user query and API call pair.

In the first pass, we used the examples given in the Problem Statement to generate a few more similar ones using GPT-3.5 and GPT-4. Further, we then feed the new tools to GPT, ask it to produce examples on similar lines as the ones produced before, and then feed the new

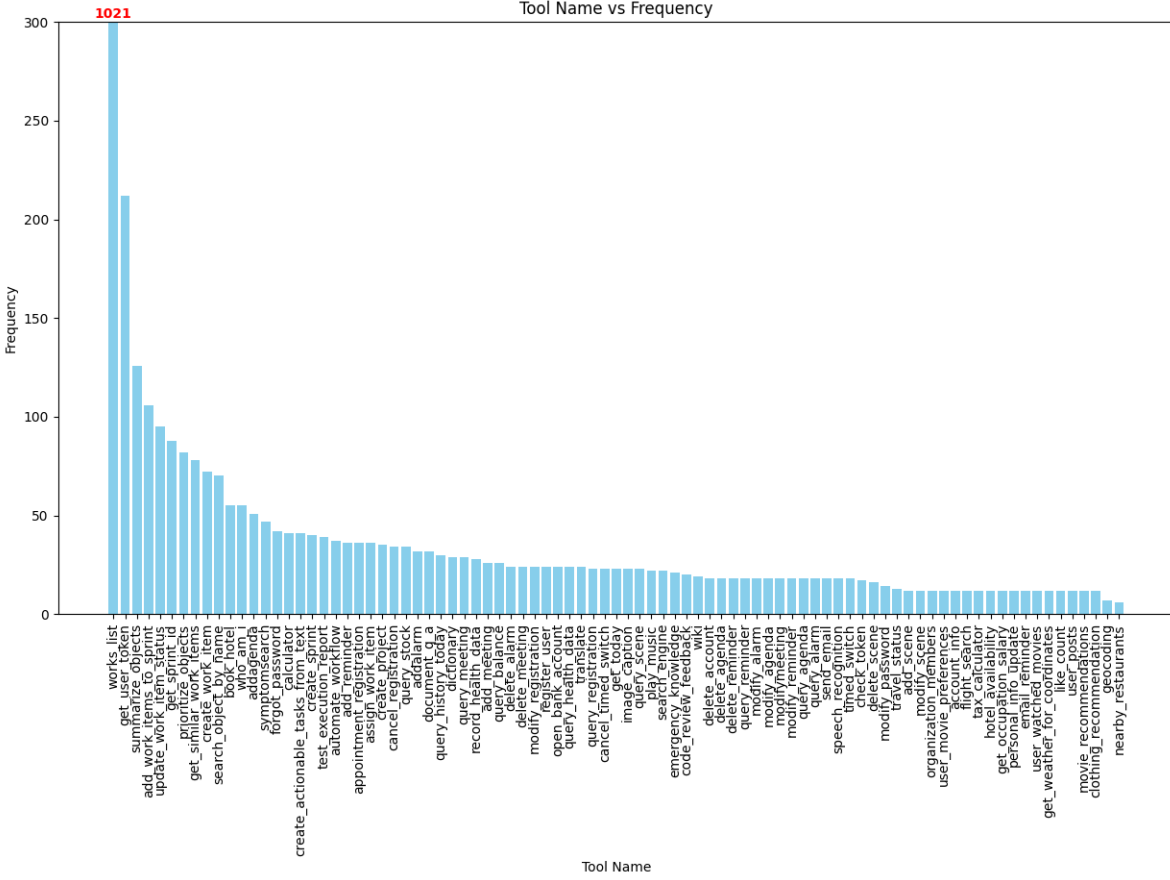


Figure 1. Number of examples utilizing each tool. We have cropped the upper limit of the graph at 300. Note that the *works_list* tool is a part of 1021 examples.

examples as well to allow GPT to rephrase and modify those examples. The prompts used for these are given in Note that this process required a tedious amount of human intervention to ensure there is no hallucination or incorrect argument names, values, or types.

Next, for the set of 71 tools recovered from [22], the examples were difficult to generate using GPT because there was some degree of hallucination involved. However, the Github Repository had several examples which were in the form of a conversation between the user and AI. Thus, we had to manually parse each conversation and summarize them into the required JSON format. Once the manual curation was done, each example was then passed through GPT to produce similar examples.

Finally, for the bonus queries, GPT was not very helpful, which required us to curate each example manually, resulting in a limited number of examples. More details for the same are given in 6.

Type of Data	Count
New Tools Added	79
Old Tools	9
Bonus Tools	4
Total Tools	92
Single Tool Single Argument	1030
Single Tool Multiple Arguments	1418
Multiple Tools	500
Tool with No Arguments	43
Empty Answers	6
Bonus queries	11
Total Examples	3008

Table 1. Categorizing all the tools and data generated

4.3. Evaluation Metrics

In assessing the effectiveness and correctness of our methods, we employ a comprehensive set of evaluation metrics tailored to the intricacies of our specific task. These

metrics offer a nuanced understanding of the model’s capabilities, covering aspects such as exact matching, precision, recall, and the propensity for hallucination.

The primary goal of our evaluation is to ensure that the model not only generates outputs that match the ground truth but also captures the subtleties inherent in the task. Given the complexity of our application, which involves predicting API calls with multiple components, a diverse set of metrics becomes imperative for a thorough evaluation.

The following are the metrics which we have used to measure the correctness or performance of our methods:

4.3.1. Exact Match

For each answer, we check if the predicted JSON is the same as the ground truth JSON, returning 1 if they match and 0 if they differ. The consideration of tool order invariance is crucial, as it assesses whether the predicted and ground truth tools maintain the same order. We allow the order to vary in the arguments of each of the tools.

While it ensures precision, EM may exhibit a limitation when semantic variations exist between predicted and ground truth JSON structures. This strict criterion might label correct predictions as false, particularly in cases where multiple valid JSON representations are possible. This highlights a consideration for scenarios where the metric’s precision may not align with the inherent variability in correct JSON outputs.

4.3.2. F_1 Score

To evaluate the similarities between two API calls, we introduce weighted F1 scores, drawing inspiration from F1 scores in context question answering tasks. Precision, recall, and F_1 are calculated for tools, arguments, and values. The overall F_1 Score is a weighted combination of these components, where the weights reflect the relative importance of each aspect. This metric provides a nuanced understanding of the model’s performance, considering both precision and recall.

The use of a weighted F1 score is motivated by the need to balance precision and recall, considering their individual importance in the context of the task.

Before we define the overall F_1 Score, we define the following,

$$\begin{aligned}\text{Precision}(x) &= \frac{\text{Total correct } x}{\text{Total predicted } x} \\ \text{Recall}(x) &= \frac{\text{Total correct } x}{\text{Total required } x} \\ F_1(x) &= \frac{2 \text{ Precision}(x) \text{ Recall}(x)}{\text{Precision}(x) + \text{Recall}(x)}\end{aligned}$$

where, $x \in \{\text{tools, arguments, values}\}$.

Now, we define the overall F_1 Score as follows,

$$F_1 = \frac{W_1 x_1 + W_2 x_2 + W_3 x_3}{W_1 + W_2 + W_3}$$

where,

$$\begin{aligned}x_1 &= F_1(\text{tools}) \\ x_2 &= F_1(\text{arguments}) \\ x_3 &= F_1(\text{values}) \\ W_1 &= 1 \\ W_2 &= \text{Recall}(\text{tools}) \\ W_3 &= \text{Recall}(\text{tools}) \text{ Recall}(\text{arguments})\end{aligned}$$

For tools, we calculate the precision, recall and F_1 by comparing the predicted and ground truth tools without checking if the order of tools is correct or not.

For arguments, we calculate the precision, recall and F_1 by comparing the predicted and ground truth arguments of correctly predicted tools.

For values, we calculate the precision, recall and F_1 by comparing the predicted and ground truth argument values of correctly predicted arguments.

Importantly, the weights (W_1, W_2, W_3) are determined by recall values. The choice of using recall values as weights implies that the importance of each component in the overall F_1 Score is influenced by how well the system recalls relevant information for that component.

If a component has higher recall, it suggests that the system is effective in capturing a larger portion of the correct instances of that component, and thus, it is considered more important in the overall evaluation.

In a contextual question-answering scenario involving API calls, the metric reflects the significance of recalling relevant tools, arguments, and values. The recall values used as weights align the evaluation with the specific requirements of the task.

The main limitation of this metric is that it does not take into account the order in which the tools are executed.

4.3.3. Hallucination Rate

This metric measures the model’s tendency to generate tools or arguments that were not present in the ground truth. Minimizing hallucination is essential for ensuring the model’s outputs align with the actual requirements.

We define hallucination rates individually for tools and arguments.

$$\text{HRate}(x) = \frac{\text{Total predicted } x \text{ not present in the tools list}}{\text{Total predicted } x}$$

where, $x \in \{\text{tools, arguments}\}$.

The overall hallucination rate is defined as,

$$\text{HRate} = \frac{\text{HRate}(\text{tools}) + \text{HRate}(\text{arguments})}{2}$$

We prefer F_1 Score over Exact Match since it captures the dynamic nature of the answers which is provided the model.

The combination of Exact Match, F_1 Score, and Hallucination Rate offers a comprehensive evaluation. Each metric provides a unique perspective on the model’s performance, and considering all three ensures a holistic assessment. This multi-faceted approach is especially valuable in capturing diverse aspects of correctness and performance.

4.4. Evaluation Metric for Information Retrieval

In our information retrieval approach, we employ a binary metric to evaluate the relevance of the retrieved tools for a given example. Initially, we extract the top k tools from the available set. Subsequently, we assess whether this set of tools encompasses all the required tools specified in a given example. If the top k tools contain all the necessary tools, we assign a value of 1; otherwise, it is marked as 0. This binary classification is applied to each example in our dataset. To quantify the overall performance, we calculate the average of these binary values by summing them up and dividing by the total number of examples. This metric provides a clear measure of the model’s ability to retrieve the most relevant tools for a diverse range of examples.

4.5. Information Retrieval Pipeline

Our Information Retrieval (IR) pipeline is designed to address the challenge presented by a dynamic set of tools and user queries within the given problem statement. The pipeline undergoes several stages to efficiently retrieve relevant tools and compose meaningful responses.

- **Initial Tool Retrieval using Information Retrieval:** At the onset, we are provided with an initial set of tools. Leveraging information retrieval techniques, we apply a ranking mechanism to identify the top 10 tools that are most relevant to a given user query. This step lays the foundation for subsequent interactions by narrowing down the set of tools for further consideration.
- **ColBERT with TF-IDF Reranking and Tool Selection:** To enhance the precision of tool retrieval, we incorporate ColBERT, a Contextualized Late Interaction model, which utilizes BERT-based contextualized embeddings. TF-IDF is then employed as a reranker to refine the initial ranking generated by ColBERT. This combined approach allows us to leverage contextual information for dynamic and context-aware tool retrieval. Subsequently, similarity scores between the tools and the user query are calculated, and the top 10 tools with the highest scores are selected as the most relevant tools for addressing the user query. This step ensures that the tools chosen align closely with

the user’s input, enhancing the effectiveness of the response.

- **Fine-tuning of ColBERT for Augmented Data:** To further improve the model’s performance and adaptability, we undertake fine-tuning of ColBERT using augmented data. Augmented data includes scenarios with added, modified, or removed tools, mimicking the dynamic nature of the real-world tool set. Fine-tuning enables the model to better generalize and handle diverse tool-related queries effectively.
- **Implementation of Add, Remove, and Modify Tool Functionality:** Recognizing the dynamic nature of the tool set, our pipeline accommodates the addition, removal, and modification of tools. Users can interactively manipulate the tool set, and our system seamlessly adapts to these changes. This functionality ensures flexibility and adaptability, allowing the model to gracefully handle dynamic tool sets.

4.6. Experiments

In this section we explained experiments that we conducted for API call generation.

4.6.1. Tool Retrieval using TF-IDF and ColBERT

4.6.1.1 Document Preparation

1. **Concatenation Approach:** In the concatenation method, we create documents by merging the tool description with detailed arguments. This results in unified documents for each tool, capturing both the tool’s overall functionality and specific details of its arguments. This holistic representation enriches the information retrieval model’s understanding, providing comprehensive context for responding to user queries.
2. **Summarization Approach using GPT-3.5:** The summarization approach involves leveraging GPT-3.5 to distill information. We pass tool descriptions through GPT-3.5 to generate concise summaries, serving as condensed representations of key tool attributes. This abstracted view enables the information retrieval model to consider high-level characteristics and essential details without the additional information present in full concatenated documents.

4.6.1.2 Training Methodology

During the training phase of the information retrieval model, we structured the training data in tuples. Each tuple consisted of three components: the first part represented the user query, the second part denoted a tool that is relevant to answering the query, and the third part indicated a tool that is not pertinent to addressing the query.

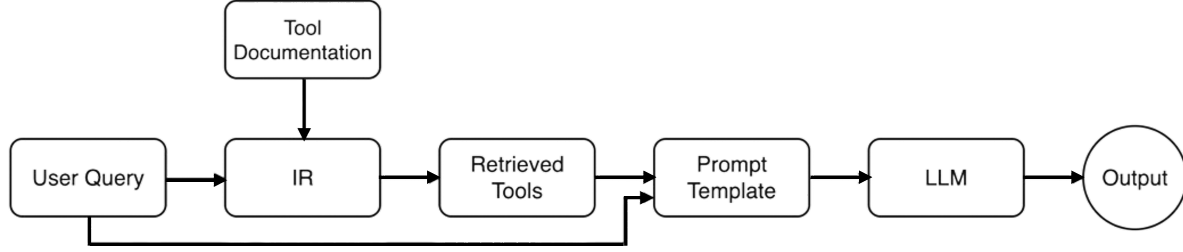


Figure 2. Overall Pipeline

This approach allowed us to create a diverse training dataset where the model learns to distinguish between tools that are contextually relevant and those that are not, enhancing its ability to accurately retrieve and recommend tools based on user queries. The utilization of these tuples in training contributes to the model’s proficiency in discerning the appropriateness of tools in response to varied queries.

4.6.2. In-context Learning for GPT Models

We use GPT-3.5 and GPT-4 [31] for in-context learning with different numbers of examples. The prompt used for the experiments is mentioned in Appendix subsection .3. Due to the cost associated with GPT models, these experiments are conducted only with 72 randomly sampled test data points.

Since API documentation is part of the text, adding new tools or updating and deleting existing ones is straightforward, as only the prompt needs to be modified. Hence, this prompt template-based setup can be generalized without any supervised fine-tuning.

4.7. Supervised Fine Tuning of Open Source LLM

We fine-tuned open source LLMs like LLaMA-2-7B, Mistral-7B and Mistral-8x-7B on 14 NVIDIA V100 GPUs 32GB each in single DGX-2 server on approx 2600 training samples for 5 epochs with initial learning rate of 2×10^{-5} and batch size 4 with fully shared data parallel with cosine linear scheduler, gradient clipping 0.7 and weight decay 0.0001, with maximum sequence length 3100.

The prompt used for this task was similar to one we used for experimentation with GPT models, except for the output format and examples provided in the prompt, as those information can be learned during supervised training. Training data was created by keeping the mixture of all relevant tools and some additional irrelevant tools as tool documentation followed by required instruction and user query. The data was further augmented by adding training samples where only irrelevant tools are provided, and the model is expected to generate empty answers as required tools are not present in tool documentation in the second augmentation setting.

During inference, the tools retrieved by the retrieval

model are used.

5. Results and Discussion

5.1. Tool Retrieval

Threshold	Accuracy
12	96.442
13	93.675
14	90.909
15	87.351
16	81.818

Table 2. Effect of Similarity Score Threshold on Tool Prediction Accuracy

The Table 2 presents the impact of varying similarity score thresholds on the accuracy of tool predictions within the top-k recommendations. Each row corresponds to a different threshold value, and the associated accuracy reflects the model’s performance when selecting tools based on this threshold. The experiment aimed to determine an optimal threshold that would yield high accuracy in tool predictions. However, it was observed that, as the threshold increased, the model predicted an excessive number of tools within the top-k range (approximately 20-30+). Despite this, the table provides valuable insights into the relationship between thresholding and prediction accuracy, contributing to a nuanced understanding of the model’s behavior under different similarity score conditions.

	Top 6	Top 8	Top 10	Avg.Time
Concatenation	93.28	95.26	98.02	0.040s
Summarization	97.62	98.41	99.20	0.044s
Summarization with 2 examples	97.62	98.41	99.20	0.043s

Table 3. Impact of Document Creation Approaches on Tool Prediction Accuracy at Different Top-k Selections. The first 3 columns are expressed as percentages

The table provides a comparative analysis of tool pre-

diction accuracy and processing time across different document preparation approaches. Specifically, the Concatenation approach and two variations of the Summarization approach are evaluated for different values of 'k' (Top 6, Top 8, and Top 10 tools).

	All Examples	Zero-Shot Examples
Approaches	Accuracy	Accuracy
Concatenation	98.42	100
Summarization	99.21	100

Table 4. Comparison of Accuracy for Tool Prediction Using Concatenation and Summarization Approaches. All values are expressed in percentages.

The Table 4 provides a comparative analysis of the accuracy achieved in tool prediction using two document preparation approaches, namely Concatenation and Summarization. The evaluation is conducted across two scenarios: "All Examples" and "Zero-Shot Examples."

5.2. GPT models with In context Learning

We have observed that GPT-4 starts to generate convincing API calls when two examples are provided. In some cases, we can observe a mismatch between the reference and predicted tools. Here, F1 refers to the F1 score. Also, the Exact Match is order-invariant for the arguments and not the order of the tools.

Model	Setting	Exact Match	F1	HR
GPT-4	0 examples	0.236	0.914	0.012
	1 example	0.250	0.908	0.006
	2 examples	0.305	0.906	0.006
GPT-3.5	0 examples	0.027	0.331	0.006
	1 example	0.236	0.826	0.007
	2 examples	0.195	0.756	0.004
	3 examples	0.285	0.691	0.006

Table 5. Results

5.3. Open Source Model

We observed that fine-tuned LLaMA-7B was still producing random tokens even after fine-tuning. However, the outputs of Mistral and Mistral-8x are correct in most cases. Table 6 contains the results of Mistral and Mistral-8x models.

	EM	F1	HR	Formatting Issues
Mistral-7B	66.79	97.29	0	3
Mistral-8X-7B	66.00	96.26	0.0073	9

Table 6. Results for Finetuned Open Source Models

In Table 7 contains a further analysis of results based on query complexity.

6. Bonus Tasks

For the bonus task, we introduced 4 new tools: *initialize_variable* , *initialize_list*, *if_tool*, and *for_each*. The *for_each* tool has arguments for *break* and *continue*, which help us emulate a *while* loop as well. The *if_tool* has also been provided with arguments for *else_if* and *else* to simulate as many conditionals scenarios as possible. To train the model, we wished to produce data using the original tools, combined with the new bonus tools we had introduced. However, due to the low quality of examples being generated by GPT-4, the examples were curated manually, with each conditional tool used in a appropriate manner. However, this limited the number of examples that we could produce.

Post generating the examples, we decided to evaluate GPT-4 on the manually created examples using a similar prompt to the one used for other examples. The evaluation was done in a zero-shot setting. We found that GPT-4 was able to generate the idea correctly, and was able to figure out the correct tools in the correct order. However, we have not quantified the performance of GPT-4 unlike the previous benchmarks. This is because there was a level of variability and subjectivity in the answers generated by GPT-4. The answers were correct if evaluated manually, but the correctness would not be reflected by a particular score. This is because the same query could be answered in different ways using different tools. An example is as below:
Question: Please check the stock prices of Amazon, Netflix, and Apple today. Today is 2023-08-09. If the price is greater than yesterday's, please send an email to abc@example.com to sell the stocks, else ask them to hold.
While the expected answer was to iterate over the list [AMZN, NFLX, AAPL] using the tool *for_each*, and then checking the stock prices for today and yesterday, GPT-4 chose to bypass the *for_each* tool, and manually check the price of stocks for each on each of the days. While both are correct, each of them has their own advantage. The advantage of using the tool *for_each* is that it allows for cleaner code and can easily be used in case we wish to check more such stocks. On the other hand, manually checking the prices allows for greater interpretability, and the usage of one tool less.

Thus, evaluating GPT-4's answers would prove tougher than just using any metric, and we believed, quantifying the performance would not do justice.

	Zero Shot Tools			Single Tool With Single Arg.			Single Tool Multiple Arguments			Multiple Tools			Empty Responses		
	EM	F1	HR	EM	F1	HR	EM	F1	HR	EM	F1	HR	EM	F1	HR
Mistral-7B	79.55	98.40	0	95.71	99.52	0	57.14	95.83	0	25	94.84	0.01	1	1	0
Mistral-8X-7B	75	97.67	0.023	94.28	99.04	0.007	58.57	93.00	0	25	94.88	0.0056	1	1	0

Table 7. Analysis of performance of finetuned models for queries with different complexity

7. Limitations of Current Approaches

Even though GPT-4 is performing second to none in few-shot settings, the current approach of few-shot prompts comes with limitations like the cost associated, and security risks for companies exposing their back-end to a third party company. Another major limitation is the context window length. None of the APIs it can handle is subjective to the context window length of the model, which can limit a simple few-shot prompt-based approach.

8. Conclusion and Future Work

For the mid-term evaluation, we worked on data augmentation for generating new API tools, and data points for the provided as well as the new tools using GPT models (GPT-3.5 and GPT-4). We further conducted benchmarking experiments using GPT models on the augmented data. As mentioned in the limitation section, the current approaches are restricted by context window size to consider more APIs, and hence, along with fine-tuning open source models like LLaMA with in-context learning, we plan to explore efficient tool retriever techniques that can provide a set of relevant tools. The retriever can help to optimize the approaches by only providing the relevant APIs. This ensures that we can reduce the prompt length significantly which directly resolves the issue of context window length in small open source LLMs like LLaMA-2. It can reduce the cost associated with input tokens in third party APIs like GPT-3.5 and 4.

References

- [1] Introduction — Langchain — python.langchain.com. https://python.langchain.com/docs/get_started/introduction. [Accessed 25-11-2023]. 3
- [2] Okapi BM25 - Wikipedia — en.wikipedia.org. https://en.wikipedia.org/wiki/Okapi_BM25. [Accessed 25-11-2023].
- [3] What is retrieval-augmented generation? — IBM Research Blog — research.ibm.com. <https://research.ibm.com/blog/retrieval-augmented-generation-RAG>. [Accessed 25-11-2023]. 4
- [4] Luiz Bonifacio, Hugo Abonizio, Marzieh Fadaee, and Rodrigo Nogueira. Inpars: Data augmentation for information retrieval using large language models, 2022. 2, 3
- [5] Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel M. Ziegler, Jeffrey Wu, Clemens Winter, Christopher Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. Language models are few-shot learners, 2020. 4
- [6] Christopher Buss, Jasmin Mousavi, Mikhail Tokarev, Arash Termehchy, David Maier, and Stefan Lee. Generating data augmentation queries using large language models. In *NeurIPS 2023 Second Table Representation Learning Workshop*, 2023. 2
- [7] Tianle Cai, Xuezhi Wang, Tengyu Ma, Xinyun Chen, and Denny Zhou. Large language models as tool makers, 2023. 4
- [8] Jiawei Chen, Hongyu Lin, Xianpei Han, and Le Sun. Benchmarking large language models in retrieval-augmented generation, 2023. 4
- [9] Wenhui Chen, Xueguang Ma, Xinyi Wang, and William W. Cohen. Program of thoughts prompting: Disentangling computation from reasoning for numerical reasoning tasks, 2023. 4
- [10] Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. Training verifiers to solve math word problems, 2021. 4
- [11] Zhuyun Dai, Vincent Y. Zhao, Ji Ma, Yi Luan, Jianmo Ni, Jing Lu, Anton Bakalov, Kelvin Guu, Keith B. Hall, and Ming-Wei Chang. Promptagator: Few-shot dense retrieval from 8 examples, 2022. 3
- [12] Qingxiu Dong, Lei Li, Damai Dai, Ce Zheng, Zhiyong Wu, Baobao Chang, Xu Sun, Jingjing Xu, Lei Li, and Zhifang Sui. A survey on in-context learning, 2023.
- [13] Dheeru Dua, Shivanshu Gupta, Sameer Singh, and Matt Gardner. Successive prompting for decomposing complex questions, 2022. 4
- [14] Luyu Gao, Aman Madaan, Shuyan Zhou, Uri Alon, Pengfei Liu, Yiming Yang, Jamie Callan, and Graham Neubig. Pal: Program-aided language models, 2023.

- [15] Wenlong Huang, Pieter Abbeel, Deepak Pathak, and Igor Mordatch. Language models as zero-shot planners: Extracting actionable knowledge for embodied agents, 2022. [3](#)
- [16] Shima Imani, Liang Du, and Harsh Shrivastava. Mathprompter: Mathematical reasoning using large language models, 2023. [4](#)
- [17] Jean Kaddour and Qi Liu. Text data augmentation in low-resource settings via fine-tuning of large language models, 2023. [2](#)
- [18] Omar Khattab and Matei Zaharia. Colbert: Efficient and effective passage search via contextualized late interaction over bert, 2020. [3](#)
- [19] Tushar Khot, Daniel Khashabi, Kyle Richardson, Peter Clark, and Ashish Sabharwal. Text modular networks: Learning to decompose tasks in the language of existing models, 2021. [4](#)
- [20] Tushar Khot, Harsh Trivedi, Matthew Finlayson, Yao Fu, Kyle Richardson, Peter Clark, and Ashish Sabharwal. Decomposed prompting: A modular approach for solving complex tasks, 2023. [4](#)
- [21] Bohan Li, Yutai Hou, and Wanxiang Che. Data augmentation approaches in natural language processing: A survey. *AI Open*, 3:71–90, 2022. [2](#)
- [22] Minghao Li, Yingxiu Zhao, Bowen Yu, Feifan Song, Hangyu Li, Haiyang Yu, Zhoujun Li, Fei Huang, and Yongbin Li. Api-bank: A comprehensive benchmark for tool-augmented llms, 2023. [2](#), [4](#), [5](#)
- [23] Yaobo Liang, Chenfei Wu, Ting Song, Wenshan Wu, Yan Xia, Yu Liu, Yang Ou, Shuai Lu, Lei Ji, Shaoguang Mao, Yun Wang, Linjun Shou, Ming Gong, and Nan Duan. Taskmatrix.ai: Completing tasks by connecting foundation models with millions of apis, 2023. [3](#)
- [24] Haokun Liu, Derek Tam, Mohammed Muqeeth, Jay Mohta, Tenghao Huang, Mohit Bansal, and Colin Raffel. Few-shot parameter-efficient fine-tuning is better and cheaper than in-context learning, 2022. [4](#)
- [25] Xiaoxia Liu, Jingyi Wang, Jun Sun, Xiaohan Yuan, Guoliang Dong, Peng Di, Wenhai Wang, and Dongxia Wang. Prompting frameworks for large language models: A survey, 2023. [2](#)
- [26] Ye Liu, Semih Yavuz, Rui Meng, Meghana Moorthy, Shafiq Joty, Caiming Xiong, and Yingbo Zhou. Exploring the integration strategies of retriever and large language models, 2023.
- [27] Pan Lu, Baolin Peng, Hao Cheng, Michel Galley, Kai-Wei Chang, Ying Nian Wu, Song-Chun Zhu, and Jianfeng Gao. Chameleon: Plug-and-play compositional reasoning with large language models, 2023. [4](#)
- [28] Xueguang Ma, Xinyu Zhang, Ronak Pradeep, and Jimmy Lin. Zero-shot listwise document reranking with a large language model, 2023. [3](#)
- [29] Grégoire Mialon, Roberto Dessì, Maria Lomeli, Christoforos Nalmpantis, Ramakanth Pasunuru, Roberta Raileanu, Baptiste Rozière, Timo Schick, Jane Dwivedi-Yu, Asli Celikyilmaz, Edouard Grave, Yann LeCun, and Thomas Scialom. Augmented language models: a survey, 2023. [2](#)
- [30] Niklas Muennighoff. Sgpt: Gpt sentence embeddings for semantic search, 2022. [3](#)
- [31] OpenAI. Gpt-4 technical report, 2023. [2](#), [8](#)
- [32] Bhargavi Paranjape, Scott Lundberg, Sameer Singh, Hannaneh Hajishirzi, Luke Zettlemoyer, and Marco Tulio Ribeiro. Art: Automatic multi-step reasoning and tool-use for large language models, 2023. [4](#)
- [33] Shishir G. Patil, Tianjun Zhang, Xin Wang, and Joseph E. Gonzalez. Gorilla: Large language model connected with massive apis, 2023. [3](#)
- [34] Ronak Pradeep, Sahel Sharifmoghaddam, and Jimmy Lin. Rankvicuna: Zero-shot listwise document reranking with open-source large language models, 2023. [3](#)
- [35] Ofir Press, Muru Zhang, Sewon Min, Ludwig Schmidt, Noah A. Smith, and Mike Lewis. Measuring and narrowing the compositionality gap in language models, 2023.
- [36] Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang, Bill Qian, Sihan Zhao, Lauren Hong, Runchu Tian, Ruobing Xie, Jie Zhou, Mark Gerstein, Dahai Li, Zhiyuan Liu, and Maosong Sun. Toolllm: Facilitating large language models to master 16000+ real-world apis, 2023. [3](#)
- [37] Zhen Qin, Rolf Jagerman, Kai Hui, Honglei Zhuang, Junru Wu, Jiaming Shen, Tianqi Liu, Jialu Liu, Donald Metzler, Xuanhui Wang, and Michael Bendersky. Large language models are effective text rankers with pairwise ranking prompting, 2023. [3](#)
- [38] Devendra Singh Sachan, Mike Lewis, Mandar Joshi, Armen Aghajanyan, Wen tau Yih, Joelle Pineau, and Luke Zettlemoyer. Improving passage retrieval with zero-shot question generation, 2023. [3](#)
- [39] Pratyusha Sharma, Antonio Torralba, and Jacob Andreas. Skill induction and planning with latent language, 2022. [3](#)
- [40] Yongliang Shen, Kaitao Song, Xu Tan, Dongsheng Li, Weiming Lu, and Yueting Zhuang. Hugginggpt: Solving ai tasks with chatgpt and its friends in hugging face, 2023. [3](#)
- [41] Weiwei Sun, Lingyong Yan, Xinyu Ma, Shuaiqiang Wang, Pengjie Ren, Zhumin Chen, Dawei Yin, and Zhaochun Ren. Is chatgpt good at search? investigating large language models as re-ranking agents, 2023. [3](#)
- [42] Qiaoyu Tang, Ziliang Deng, Hongyu Lin, Xianpei Han, Qiao Liang, Boxi Cao, and Le Sun. Toolalpaca: Generalized tool learning for language models with 3000 simulated cases, 2023. [3](#)
- [43] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, Aurelien Rodriguez, Armand Joulin, Edouard Grave, and Guillaume Lample. Llama: Open and efficient foundation language models, 2023. [2](#)
- [44] Yizhong Wang, Yeganeh Kordi, Swaroop Mishra, Alisa Liu, Noah A. Smith, Daniel Khashabi, and Hannaneh Hajishirzi. Self-instruct: Aligning language models with self-generated instructions, 2023. [2](#)
- [45] Tomer Wolfson, Mor Geva, Ankit Gupta, Matt Gardner, Yoav Goldberg, Daniel Deutch, and Jonathan Berant. Break it down: A question understanding benchmark, 2020. [3](#)
- [46] Qiantong Xu, Fenglu Hong, Bo Li, Changran Hu, Zhengyu Chen, and Jian Zhang. On the tool manipulation capability of open-source large language models, 2023. [2](#)

- [47] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models, 2023.
- [48] Xinyu Zhang, Sebastian Hofstätter, Patrick Lewis, Raphael Tang, and Jimmy Lin. Rank-without-gpt: Building gpt-independent listwise rerankers on open-source large language models, 2023. [3](#)
- [49] Denny Zhou, Nathanael Schärli, Le Hou, Jason Wei, Nathan Scales, Xuezhi Wang, Dale Schuurmans, Claire Cui, Olivier Bousquet, Quoc Le, and Ed Chi. Least-to-most prompting enables complex reasoning in large language models, 2023. [3](#)
- [50] Fengbin Zhu, Wenqiang Lei, Chao Wang, Jianming Zheng, Soujanya Poria, and Tat-Seng Chua. Retrieving and reading: A comprehensive survey on open-domain question answering, 2021. [2](#)

.1. Appendix A

```
[
  {
    "question_id": "2",
    "question": "Summarize high severity tickets from the customer UltimateCustomer",
    "answer": [
      {
        "tool_name": "search_object_by_name",
        "arguments": [
          {
            "argument_name": "query",
            "argument_value": "UltimateCustomer"
          }
        ]
      },
      {
        "tool_name": "works_list",
        "arguments": [
          {
            "argument_name": "ticket.rev_org",
            "argument_value": ["$$PREV[0]"]
          },
          {
            "argument_name": "ticket.severity",
            "argument_value": ["high"]
          },
          {
            "argument_name": "type",
            "argument_value": ["ticket"]
          }
        ]
      }
    ],
    {
      "tool_name": "summarize_objects",
      "arguments": [
        {
          "argument_name": "objects",
          "argument_value": "$$PREV[1]"
        }
      ]
    }
  ]
]
```

```
}
]
```

Listing 1. An illustrative Question-Answer Pair in JSON Format

```
[
  {
    "tool_name": "
      add_work_items_to_sprint",
    "tool_description": "Adds the given
      work items to the sprint",
    "arguments": [
      {
        "name": "work_ids",
        "description": "A list of
          work item IDs to be
          added to the sprint",
        "type": "array of strings",
        "example": ""
      },
      {
        "name": "sprint_id",
        "description": "The ID of
          the sprint to which the
          work items should be
          added",
        "type": "str",
        "example": ""
      }
    ]
  }
]
```

Listing 2. Representation of Provided Tools in JSON Format

.2. Appendix B

```
[
  {
    "question_id": "2",
    "question": "Summarize high severity
      tickets from the customer
      UltimateCustomer",
    "answer": [
      {
        "tool_name": "
          search_object_by_name",
        "arguments": [
          {
            "argument_name": "query",
            "argument_value": "
              UltimateCustomer"
          }
        ]
      },
      {
        "tool_name": "works_list",
```

```
      "arguments": [
        {
          "argument_name": "
            ticket.rev_org",
          "argument_value":
            ["$$PREV[0]"]
        },
        {
          "argument_name": "
            ticket.severity",
          "argument_value":
            ["high"]
        },
        {
          "argument_name": "
            type",
          "argument_value":
            ["ticket"]
        }
      ]
    },
    {
      "tool_name": "
        summarize_objects",
      "arguments": [
        {
          "argument_name": "
            objects",
          "argument_value": "
            $$PREV[1]"
        }
      ]
    }
  ]
}
```

Use the above example and generate 5 more diverse examples in which the question is reframed keeping the semantics constant. Also, change the argument values of the question according to the question. The question and argument values should use real-life human names. Only output the examples, no extra details should be printed. Generate the examples in one JSON file.

\subsection{Appendix D} \label{appendix_d}

\begin{lstlisting}[style=json, caption={Prompt template for benchmarking experiments on GPT models}, label={lst:gpt_Prompt}]

Use the following tools and answer the given question using given answer format


```

{{ API DOCUMENTATION in JSON format}}

Generated answers for any given question
should have following format

{{ ANSWER FORMAT PROVIDED IN PROBLEM
STATEMENT}}

Following are the examples of question-
answer pairs for the abovementioned
tools

EXAMPLES FOR REFERENCE in case of FEW SHOT
SETTING

To reference the value of the ith tool in
the chain, use $$PREV[i] as argument
value. i = 0, 1, .. j-1; j = current
tools index in the array
If the query could not be answered with the
given set of tools, output an empty
list instead.

Empty list should look like []. Whenever
you want to access information about
current user to pass it further, you
need to make a call to who_am_i api tool
.

Answer the following question based on the
instructions provided above.
question:
answer:{
"question":
"answer":
}

```

Listing 3. Example of a prompt to generate multiple datapoints from a single example

.3. Appendix C

```

Use the following tools and answer the
given question using given answer format

{{ API DOCUMENTATION in JSON format}}

Generated answers for any given question
should have following format

{{ ANSWER FORMAT PROVIDED IN PROBLEM
STATEMENT}}

Following are the examples of question-
answer pairs for the abovementioned

```

```

tools

EXAMPLES FOR REFERENCE in case of FEW SHOT
SETTING

To reference the value of the ith tool in
the chain, use $$PREV[i] as argument
value. i = 0, 1, .. j-1; j = current
tools index in the array
If the query could not be answered with the
given set of tools, output an empty
list instead.

Empty list should look like []. Whenever
you want to access information about
current user to pass it further, you
need to make a call to who_am_i api tool
.

Answer the following question based on the
instructions provided above.
question:
answer:{
"question":
"answer":
}

```

Listing 4. Prompt template for benchmarking experiments on GPT models